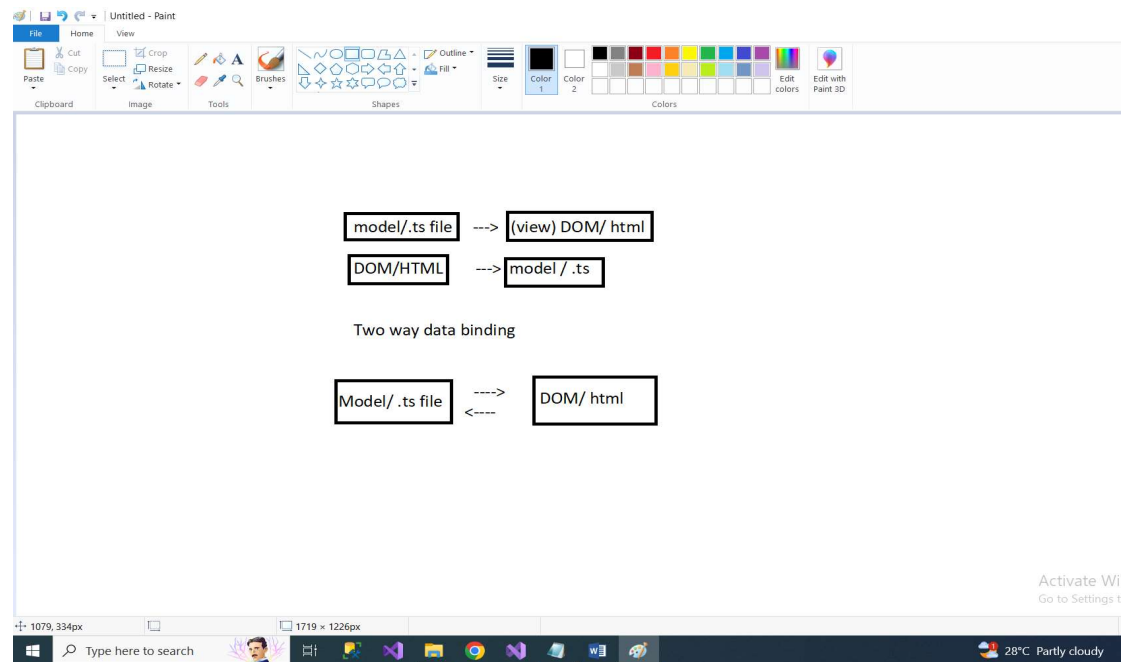


Data Binding

It is used to send between files.

One way data binding:



One way data binding
string interpolation
attribute (property) binding
class binding
style binding
event binding

string interpolation :

It is one way binding. Sends data from type script to html file. Synchronization will works where i.e., when changes done in typescript file then changes are effected in html file.

syntax:

```
{{ expression }}
```

app.ts:

```
export class App {  
  x=1  
  fun1()  
}
```

```

        {
            return this.x
        }
    }
}

```

app.html:

```

<h1> {{ title }} </h1>
<h1> {{ fun1() }} </h1>
<h1> {{ 60*2 }} </h1>

```

ex 2 :

app.ts:

```

export class App
{
    city:string="Hyderabad"
}

```

app.html:

```

<h1> {{ city }} </h1>

```

ex:3

create assets folder in src folder. Copy image in assets folder.

Angular.json file:

```

"assets": [
    "src/favicon.ico",
    "src/assets"
],

```

app.ts:

```

export class App
{
    path:string="assets/image1.jpg"
}

```

app.html:

```

<h1> <img src={{path}} alt="no image"> </h1>

```

ex:4

app.css:

```

.special
{

```

```
        border: 3px solid green;
    }
}
```

app.html:

```
<h1 class="special"> Sathya Technologies </h1>
```

ex:5

app.ts:

```
export class App
{
    cname:string="special"
}
```

app.css:

```
.special
{
    border: 3px solid green;
}
```

app.html:

```
<h1 class={{cname}}> Sathya Technologies </h1>
```

ex:6

app.ts:

```
export class App
{
    d:any = new Date().toLocaleDateString()
}
```

app.html:

```
<h1 > {{d }} </h1>
```

Property Binding:

It is one way data binding. Allows to bind data from typescript file (model) to html. html attributes here they are called properties.

sytnax:

```
[property]
```

ex:1

app.ts:

```
export class App
{
    isdisabled:boolean=true
}
```

app.html:

```
<button [disabled]=isdisabled >Submit </button>
```

ex:2

app.ts:

```
export class App
{
    ishidden:boolean=true
}
```

app.html:

```
name : <input type="text" [hidden]=ishidden>
```

ex:3

create assets folder in src folder. Copy image in assets folder.

Angular.json file:

```
"assets": [
  "src/favicon.ico",
  "src/assets"
],
```

app.ts:

```
export class App
{
    path:string='assets/image1.jpg'
}
```

app.html:

```
<img src={{path}} alt="no image">
    or
<img [src]="path" alt="no image">
```

Class Binding:

ex:1

app.ts:

```
export class App
{
    isactive:boolean=true;
}
```

App.css:

```
.active{
    background-color:blue;
    color:white;
}
.inactive{
    background-color:gray;
    color:white;
}
```

app.html:

```
<h1 [class]="isactive?'active':'inactive' "> Sathya Technologies </h1>
```

Note : ? : is called ternary operator

ex:2 using multiple classes based on multiple condition.

app.ts:

```
export class App
{
    isactive:boolean=true;
    applyC1:boolean=true;
}
```

app.css:

```
.active{
    background-color:blue;
    color:white;
}
.inactive{
    background-color:gray;
    color:white;
}
.c1{
    border:5px solid green;
}
```

app.html:

```
<h1 [class]="{c1:applyC1,active:isactive}"> Sathya Technologies </h1>
or
<h1 [class]="['c1','active'] "> Sathya Technologies </h1>
```

or
`<h1 [class.active]=isactive> Sathya Technologies </h1>`

Style Binding

it is one way data binding. It is send value dynamically to inline style.

syntax:

`[style]={property:variable_name}`
or
`[style.property]=variable_name`

ex: 1

app.ts:

```
export class App
{
    cvar:string='gray'
}
```

app.html:

`<h1 [style]="{color:cvar}"> Sathya Technologies </h1>`
or
`<h1 [style.color]="cvar"> Sathya Technologies </h1>`

ex: 2

app.ts:

```
export class App
{
    mystyle:object={
        color:'white',
        background:'gray',
        border:'5px solid blue'
    }
}
```

App.html:

`<h1 [style]="mystyle"> Sathya Technologies</h1>`

ex 3:

app.ts:

```
export class App
{
    hasError:boolean=true
}
```

app.html:

```
<h1 [style.color]="hasError?'red':'green' "> Sathya Technologies</h1>
```

Event Binding (Event handling):

Event binding is **used to handle the events raised by the user actions** like button click, mouse movement, keystrokes, etc. It is one way data binding. Allows to bind data from html to model (typescript file)

syntax:

```
(eventname) = function_name()  
or  
on-eventname = function_name()
```

mouse events:

- click
- double click
- mouse over
- mouse out
- mouse move

keyboard events:

- key down
- key up
- key press

focus events:

- focus
- blur
- focus in
- focus out

form events:

- submit
- reset
- change

ex:1

app.ts:

```
export class App  
{  
    counter:number=0;  
    fun1(){  
        this.counter+=1;  
    }  
}
```

```

        fun2(){
            this.counter-=1;
        }
    }

```

app.html:

```

<button (click)="fun1()"> Increment </button>
<button (click)="fun2()"> Decrement </button>
<h1> Counter : {{ counter }} </h1>

```

Ex :2

app.ts:

```

export class App
{
    changeName(e:any) {
        console.log(e)
        console.log(e.target)
        console.log(e.target.value)
    }
}

```

app.html:

```

Name: <input type="text" (input)="changeName($event)">

```

Note: "input" is the event. This is executed for every character given in textbox.

- e - property shows input event
- target - property will show <input> textbox tag.
- Value - property will the character given in textbox

\$event:

In Angular we can use the \$event variable. This variable will provide information about the event that was invoked. Sometimes passing the event object might not be very useful. Instead, you may want to pass the value of an input on the page.

Two-way Binding

Data modified in typescript should be updated in html file. And data modified in html file should be updated in typescript file.

syntax:

```

[(ngModel)]=propertyname(variable)

```

using html forms we can update dynamic data.

app.ts:

```

import { FormsModule } from '@angular/forms'

@Component({
  - -
  imports: [RouterOutlet, FormsModule],
  - -
})

export class App {
  city="Hyderabad"

  fun1(){
    this.city="Bangalore"
  }
}

```

app.html

```

city: <input type="text" [(ngModel)]="city">
<p> {{ city }} </p>
<button (click)="fun1()">Update</button>

```

Note: The ng-model directive binds the value of HTML controls (input, select, textarea) to application data.